PromptFlow: Comprehensive Engineering Documentation

Autonomous Micro-Automation Engine via Natural Language Compilation

1. Executive Summary & Abstract

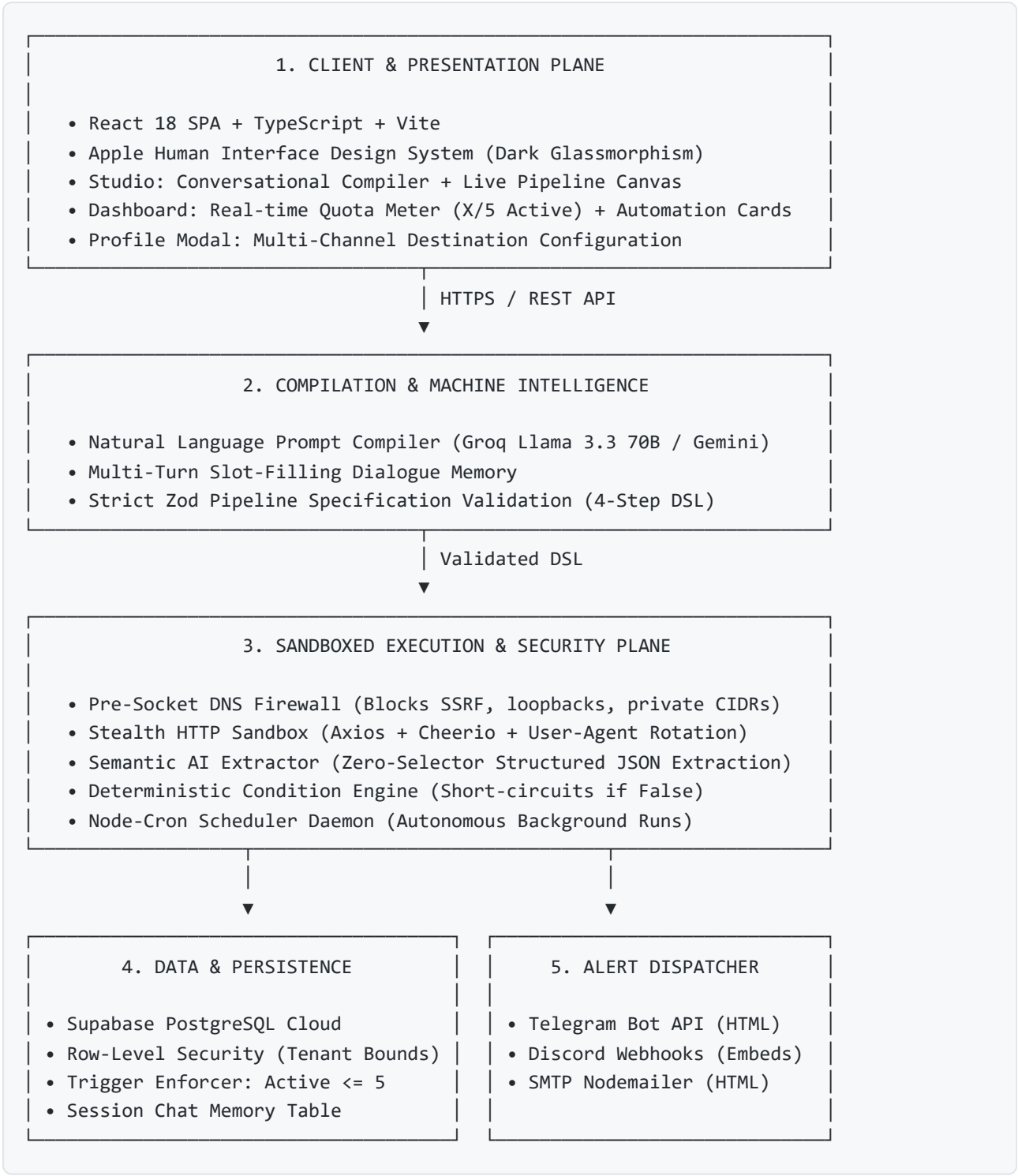
PromptFlow is a modern, autonomous micro-automation engine designed to bridge human natural language intent with deterministic web extraction and real-time multi-channel notification delivery.

Traditional web scrapers and automation tools (e.g., Selenium, Playwright, Zapier, Make.com) suffer from three critical problems: 1. **High Technical Barrier:** Users must master programming languages, regular expressions, XPath strings, or complex 2D node graphs. 2. **DOM Fragility:** Static CSS selectors break silently the moment a target website updates its class names, layout structure, or A/B testing variants. 3. **Security Vulnerabilities:** Naive scraping engines expose internal infrastructure to Server-Side Request Forgery (SSRF) attacks when arbitrary user URLs are fetched without strict network-level sanitization.

PromptFlow solves this through a **Hybrid Intelligence Architecture**: * **Semantic Understanding Plane:** Large Language Models (LLMs) interpret user intent, conduct conversational slot-filling to resolve ambiguities, and extract structured data from raw web HTML without brittle CSS selectors. * **Deterministic Execution Plane:** An isolated, sandboxed runtime executes HTTP fetches, applies a Pre-Socket DNS Firewall, evaluates conditional triggers with mathematical rigor, and dispatches push notifications via Telegram, Discord, or Email.

2. System Architecture & High-Level Design

PromptFlow is decoupled into four primary architectural planes:



3. The 4-Step Pipeline Domain Specific Language (DSL)

PromptFlow organizes every micro-automation into an immutable, 4-step execution graph:

$$\text{Step 1: Scrape} \longrightarrow \text{Step 2: AI Filter} \longrightarrow \text{Step 3: Condition} \longrightarrow \text{Step 4: Notify}$$

DSL Specification Schema

```
export interface PipelineSpecification {
  title: string;
  description?: string;
  schedule_cron: string; // Standard 5-field cron (e.g. "0 9 * * *")
  steps: [
    StepScrape,
    StepAIFilter,
    StepCondition,
    StepNotify
  ];
}
```

Detailed Step Breakdown

Step	Type	Purpose	Configuration Parameters
1	`scrape`	Fetches raw HTML from target URL within an isolated sandbox.	`url` : Validated HTTP/HTTPS URL `max_length` : Max text payload length (default: 5000 chars)
2	`ai_filter`	Extracts target entities from raw page text into structured JSON.	`instruction` : Natural language extraction rule `expected_fields` : Array of expected keys (e.g. `["title", "price"]`)
3	`condition`	Evaluates extracted values against mathematical or string rules.	`field` : Dot-notation path (e.g. `step_2.price`) `operator` : `==` , `!=` , `>` , `<` , `>=` , `<=` , `contains` `value` : Threshold or match target
4	`notify`	Dispatches formatted push alert if Step 3 evaluates to true.	`channel` : `"telegram"` | `"discord"` | `"email"` `message` : Template string with token interpolation (`{{step_2.title}}`)

---

4. Machine Intelligence & Compilation Engine

4.1 Natural Language Compilation

When a user submits a prompt, the backend compiles it into a validated 4-step pipeline: 1. **Model Selection:** Primary routing targets **Groq Cloud** running `llama-3.3-70b-versatile` with low latency

($\approx 800\text{ms}$). 2. **Fallback Circuit:** If Groq encounters rate-limiting or network timeout, the request seamlessly fails over to **Google Gemini 1.5/2.0 Flash**. 3. **Schema Enforcement:** The LLM's response is parsed and strictly validated against Zod schemas. If the LLM generates invalid JSON, PromptFlow runs an internal retry loop.

4.2 Multi-Turn Slot-Filling Dialogue

If a user prompt lacks required information (such as target URL, numerical threshold, or channel), PromptFlow does not guess or fail. It enters an `incomplete` state: * Identifies missing slots: `["target_url", "threshold"]`. * Maintains conversation history in the Supabase `chat_history` table keyed by `session_id`. * Generates a contextual clarification question:

> *"Could you provide the product page URL and the price threshold you'd like to monitor?" * When the user answers, the compiler merges previous turns and produces the final compiled pipeline.*

---

5. Stealth Scraping Sandbox & Semantic Extraction

5.1 Stealth Network Sandbox

- **User-Agent Rotation:** Emulates modern Chrome/Blink browsers on macOS and Windows to prevent trivial bot detection.
- **Payload Sanitation:** Strips `<script>`, `<style>`, `<svg>`, `<nav>`, and base64 media from raw HTML using Cheerio, minimizing token consumption and eliminating injection vectors.
- **WAF & 403 Challenge Translation:** When target websites return HTTP 403 (e.g. Cloudflare bot protection), PromptFlow marks Step 1 with a structured diagnostic banner offering actionable user guidance.

5.2 Zero-Selector Semantic AI Extraction

Traditional scrapers depend on CSS classes like `.price-current-value_34a`. When the site undergoes an A/B test or updates CSS modules, the scraper breaks. PromptFlow passes clean text excerpts to the LLM extraction prompt:

```
Extract the requested fields from the text. Return ONLY a valid JSON object matching:
Expected fields: ["title", "price"]
Text: [Page content...]
```

The model understands synonyms, varied formatting, and currency symbols, extracting clean primitives (`price: 51.77`) regardless of DOM restructuring.

---

6. Security Architecture & Pre-Socket SSRF Firewall

A critical vulnerability in user-facing web scrapers is **Server-Side Request Forgery (SSRF)**. Malicious users may attempt to input URLs like: * `http://localhost:5000/api/admin` * `http://127.0.0.1:8080/internal-metrics` * `http://169.254.169.254/latest/meta-data/` (AWS Cloud Credentials) * `http://192.168.1.1/router-settings`

Pre-Socket DNS Inspection Mechanism

PromptFlow intercepts URLs *prior* to opening any TCP connection: 1. **Protocol Validation:** Restricts protocols strictly to `http:` and `https:`. 2. **DNS Pre-Resolution:** Resolves the hostname via Node.js `dns/promises.lookup`. 3. **Subnet Inspection:** Inspects resolved IPv4 and IPv6 addresses against private and reserved CIDR ranges: * Loopback: `127.0.0.0/8`, `::1` * Private Class A: `10.0.0.0/8` * Private Class B: `172.16.0.0/12` * Private Class C: `192.168.0.0/16` * Link-Local / Cloud Metadata: `169.254.0.0/16` 4. **Immediate Rejection:** If an address resolves to a restricted subnet, the request is blocked instantly with: `SSRF Prevention: Host "X" resolves to private address "Y".`

---

7. Deterministic Evaluation & Short-Circuit Engine

Step 3 acts as the gatekeeper before alert dispatch: * **Currency Sanitization:** Automatically parses currency symbols (`$`, `£`, `€`, `¥`) and formats strings into numbers (`"£51.77"` $\rightarrow$ `51.77`). * **Supported Operators:** * Numeric: `<`, `>`, `<=`, `>=`, `==`, `!=` * String / Substring: `contains` (case-insensitive) * **Short-Circuit Logic:** If the condition evaluates to `false` (e.g., current price £51.77 is not $< \text{£}40$), the pipeline transitions immediately to `SKIPPED`: * Step 4 (`notify`) is **not executed**. * No external notification is dispatched. * Server resources and API quotas are preserved.

---

8. Multi-Channel Alert Dispatcher

When Step 3 evaluates to `true`, PromptFlow resolves message template tokens (`{{step_2.title}}`, `{{step_2.price}}`) and delivers alerts across three supported channels:

8.1 Telegram Bot API

- Uses official Telegram Bot API `sendMessage` endpoint.
- Parses HTML tags (`<b>`, `<code>`, `<i>`).
- Dispatches messages directly to user's private chat ID or group.

8.2 Discord Webhooks

- Formats alerts into rich embeds featuring:
- Embed title: `⚡ PromptFlow Alert`
- Theme color: Sky Blue (`#0284c7`)
- Dynamic timestamp & footer.

8.3 Email (SMTP / Nodemailer)

- Dispatches responsive, dark-mode HTML email with customized header and structured bullet lists.
- Supports Gmail App Passwords, AWS SES, or standard SMTP relays.

---

9. Database Architecture & Quota Enforcement

9.1 PostgreSQL Schema (Supabase)

- **`profiles`:** Stores user settings, full name, Telegram Chat ID, and Discord Webhook URL.
- **`automations`:** Stores pipeline specifications, active status (`is_active`), cron schedules, and ownership (`user_id`).
- **`execution_logs`:** Records run history, duration in milliseconds, status (`SUCCESS` , `SKIPPED` , `FAILED`), and error details.
- **`chat_history`:** Stores multi-turn session messages for conversational slot-filling.

9.2 Strict 5-Slot Quota Trigger

To prevent resource abuse, a PostgreSQL database trigger actively enforces capacity at the database level:

```
CREATE OR REPLACE FUNCTION enforce_active_automation_limit()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.is_active = true THEN
    IF (SELECT COUNT(*) FROM automations WHERE user_id = NEW.user_id AND is_active = true A
      RAISE EXCEPTION 'Active automation quota exceeded: Maximum 5 concurrent active automa
    END IF;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

10. Frontend Architecture & Apple Design System

The PromptFlow client is built with React 18 and Tailwind CSS, adhering strictly to **Apple Human Interface Guidelines (HIG)**: * **Dark Glassmorphism:** Deep obsidian base (`#05070D`), translucent glass panels (`rgba(255,255,255,0.07)`), and backdrop blur filters (`blur(28px) saturate(170%)`). * **Ambient Status Glows:** Real-time visual feedback on Step Cards: * Success: Intense Apple neon green glow (`0 0 28px rgba(48,209,88,.35)`). * Blocked / Failed: Vivid crimson glow (`0 0 28px rgba(255,69,58,.35)`). * **Spring Physics:** Custom cubic-bezier transition curves (`cubic-bezier(0.16, 1, 0.3, 1)`) for modal popups, drawers, and buttons. * **Ergonomic Viewports:** Studio and Dashboard views are centered vertically and horizontally on standard laptop viewports, eliminating awkward window-level scroll jumps.

11. Production Cloud Deployment

Component	Platform	URL / Host	Configuration
Frontend	**Vercel**	`https://prompt-flow-ebon.vercel.app/`	Vite preset, `vercel.json` SPA rewrites
Backend	**Render**	`https://promptflow-31p9.onrender.com`	Node 20+, persistent cron daemon
Database	**Supabase**	`yeimehtqdcmpxvgngruj.supabase.co`	PostgreSQL 15, Auth, RLS policies

12. Automated Verification & Testing Suite

PromptFlow includes two automated test suites guaranteeing 100% verification across all branches:

Root E2E Test Suite (`node test.js`)

- **Test 1:** Backend API Health Check $\rightarrow$ `PASS`
- **Test 2:** Frontend Asset Delivery $\rightarrow$ `PASS`
- **Test 3:** AI Compiler Pipeline Synthesis $\rightarrow$ `PASS`
- **Test 4:** Slot-Filling Ambiguity Dialogue $\rightarrow$ `PASS`
- **Test 5:** Sandbox Scraping & Extraction $\rightarrow$ `PASS`
- **Test 6:** False Condition Short-Circuiting $\rightarrow$ `PASS`

- **Test 7:** SSRF Pre-Socket Firewall Enforcement $\rightarrow$ `PASS`
- **Test 8:** Multi-Channel Bullet Formatting $\rightarrow$ `PASS`
- **Test 9:** Capacity Quota Governance (5-Active Limit) $\rightarrow$ `PASS`

Overall Verification Score

$$\text{Total Features Tested: } 9 \quad\mid\quad \text{Passed: } 9 \quad\mid\quad \textbf{Pass Rate: 100\%}$$

---

13. API Reference Summary

Endpoint	Method	Description	Payload / Parameters
`/api/health`	`GET`	Health status and engine uptime	None
`/api/compile`	`POST`	Compiles plain English prompt into 4-step DSL	`{ prompt: string, session_id?: string, user_id?: string }`
`/api/execute/test`	`POST`	Executes dry run in sandbox	`{ pipeline: PipelineSpecification, user_destinations?: object }`
`/api/automations`	`POST`	Saves and activates automation	`{ user_id: string, pipeline: PipelineSpecification, is_active: boolean }`
`/api/automations/:id/toggle`	`PATCH`	Pauses or resumes an automation	`{ is_active: boolean }`
`/api/automations/:id`	`DELETE`	Deletes automation	None

Author: Himanshu Lokhande · PromptFlow Project Documentation · Production Release 1.0